Loci: A Spatial Memory Agent on Jetson Orin Nano

Oct 9, 2026 · @Ram Siya Ram

Flagship project proposal for NVIDIA and Bay Area entry-level roles, researched October 2026.

The pitch

Loci gives AI agents a memory of the physical world, running entirely on your Jetson Orin Nano Super and RealSense D415. It watches a room or workbench, remembers what it saw and where (in meters), and answers plain-English questions or enforces watch rules. No video ever leaves the board.

The name comes from the method of loci, the memory-palace technique of storing memories by place, which is exactly what the system does.

The 90-second demo (outputs below are examples, to be replaced with real ones):

1. **Ask:** "Where did I leave my keys?" returns "On the desk, left of the monitor, about 1.4 m from the camera, last seen 3:42 pm" plus the keyframe.
2. **Watch:** "Alert me if someone stays within 1 m of the 3D printer for over 10 seconds" compiles into an on-device rule; it fires live, and the VLM confirms before the alert goes out.
3. **Measure:** "How big is this box?" returns width, depth and height from the depth camera.
4. **Integrate:** an outside agent (Claude, or a NeMo Agent Toolkit agent) connects over MCP and asks "What changed on the bench since 9 am?"
5. **Prove it:** a jtop overlay shows everything running locally, with live power draw.

It stands out because it is a system, not a model demo: a real-time C++/CUDA loop, metric 3D grounding, a memory store, a scheduler, an agent and an open protocol, all fitted into 8 GB with measured tradeoffs.

Why it gets you hired

Each requirement in NVIDIA's 2026 new-grad postings that fits your background maps to a concrete artifact in this repo.

What the postings ask for	Where Loci proves it
C++, Python and CUDA to deploy LLMs and generative AI (ML Inference NCG, TensorRT team)	C++ perception service, custom CUDA kernels, VLM served by TensorRT Edge-LLM's C++ runtime
Benchmarking methods, quantization, scheduling and memory management for TensorRT and TensorRT-EdgeLLM (TensorRT Performance NCG)	Published Orin Nano benchmarks, INT8 vs FP16 detector study, INT4 VLM, memory budget, priority scheduler
C++ with object-oriented design, Docker, CMake, CI (Deep Learning Libraries NCG)	Modular C++ built with CMake, unit tests, Docker images, GitHub Actions

What the postings ask for	Where Loci proves it
Strong C/C++, software design and AI algorithms (DevTech AI NCG)	Kernels profiled in Nsight Systems, written up like a DevTech blog post
Edge AI and robotics platforms (Embedded and Robotics FAE NCG, Isaac roles)	JetPack 7.2, Cosmos Reason 2, physical-AI framing, a public Jetson community post

It also turns your resume into one storyline instead of a list. SentrySight's dual-model agreement becomes detector-plus-VLM verification, and your production agentic RAG becomes RAG over physical-world memory. Your Monday.com MCP integration becomes an MCP server, your RealSense dimensioning becomes metric 3D grounding, and your FlashAttention kernel becomes production CUDA profiled end to end.

One targeting note: NCG postings are tied to a graduation year, and the 2027 postings seen so far ask for 2027 graduates (example). As a December 2025 grad, prioritize still-open NCG 2026 roles and roles listing 0-2 years of experience, and confirm eligibility on each posting.

Hardware reality check

Memory, not TOPS, is the constraint that shapes this design: the board has 8 GB shared by CPU and GPU, and the same 2B VLM peaks at 4.3 GB or 6.9 GB depending on the runtime.

Your board. Orin Nano Super Developer Kit, 8 GB, on JetPack 6.2 from the SD card (L4T 36.4.7) per your July setup notes, with a 256 GB NVMe installed but not yet set up, plus your RealSense D415.

Verified specs (NVIDIA): 67 INT8 TOPS; Ampere GPU with 1024 CUDA cores and 32 Tensor Cores; 6-core Cortex-A78AE at up to 1.7 GHz; 8 GB 128-bit LPDDR5 at 102 GB/s; 7-25 W.

Software to build on, as of October 2026:

- JetPack 7.2 moves Orin onto the JetPack 7 stack: Jetson Linux 39.2, kernel 6.8, Ubuntu 24.04, CUDA 13.2.1, TensorRT 10.16.2, DeepStream 9.1, Nsight Systems 2026.3. The Orin Nano dev kit no longer has an SD card image; it installs from an ISO on a USB stick, which suits your unused NVMe. JetPack 7.2.1 flashes the dev kit in Super mode by default.
- TensorRT Edge-LLM is NVIDIA's C++ runtime for LLMs and VLMs on Jetson, with no Python in the inference path. On Orin it requires JetPack 7.2 and runs FP16 or INT4 AWQ (FP8 and NVFP4 are Thor-only). Its export step runs on an x86 NVIDIA GPU or a Thor, not on the Orin.
- Cosmos Reason 2 2B is NVIDIA's compact VLM with chain-of-thought and spatial reasoning. Jetson AI Lab recommends llama.cpp on the Orin Nano; its vLLM recipe for the Nano needs tight limits, such as a 768-token context and one image per prompt.
- DeepStream 9.1 is the first release that targets JetPack 7.2 on Orin, but its VLM plugin is documented for x86 GPUs only. On-device VLM work therefore goes through Edge-LLM or llama.cpp.
- librealsense documents an RSUSB build that skips kernel patching and covers Ubuntu 24.04 / kernel 6.8 images. Orin Nano on JetPack 7.2 is not listed as verified, so test it first.

Measured by others (one developer's benchmark repo, JetPack 6.2.2, MAXN SUPER). Treat these as a baseline to reproduce, not a promise.

Workload	Runtime	Measured
YOLOv8n, 640x640	TensorRT	4.43 ms FP16, 3.49 ms INT8 (GPU time only)
Cosmos-Reason2-2B, 4-bit	llama.cpp	38 tokens/s; 114 ms to first token with an image
Cosmos-Reason2-2B, 4-bit	vLLM	56 tokens/s; 75 ms to first token with an image; 6.9 GB peak
Cosmos-Reason2-2B, 4-bit	TensorRT Edge-LLM	60 tokens/s; 420 ms to first token with an image; 4.3 GB peak

On JetPack 6.2.2 the Edge-LLM path needed two workarounds, a CMA kernel parameter and an ONNX graph rewrite. Whether JetPack 7.2 still needs them is an open question you will answer and publish.

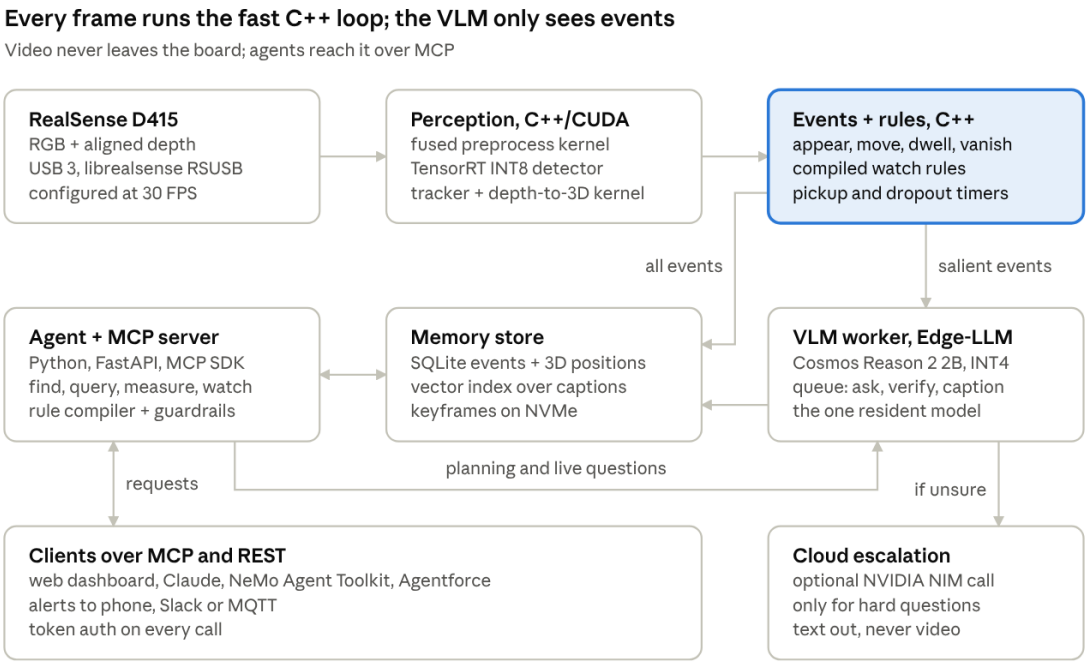
GPU-only timings also hide pipeline cost. One forum report shows 3.8 ms preprocess and 8.0 ms postprocess around a 13.5 ms inference in Python, which is exactly what a C++/CUDA path removes.

Memory budget (targets; measure each with tegrastats in week 1):

Component	Plan	Budget
OS, headless	Jetson AI Lab RAM optimization steps, no desktop	measure; target 1.5 GB or less
VLM, eyes and agent brain	Cosmos-Reason2-2B INT4 on Edge-LLM	4.3 GB peak, measured by others
Detector, tracker, frame buffers	TensorRT INT8 engine, pinned zero-copy buffers	estimate 0.3-0.5 GB
Embeddings and vector index	small text-embedding model over captions	estimate 0.2-0.5 GB
Agent, API, MCP server	one Python process	estimate 0.2-0.3 GB
Headroom	KV-cache growth, spikes	keep at least 0.5 GB

The budget only closes because one model does both the seeing and the reasoning. A second resident LLM would not fit, and that is the first design decision to defend in interviews.

Architecture



Loci system architecture · fast loop, slow loop, agent interface

Read it left to right, then down: every event lands in memory, only salient ones reach the VLM, and the agent answers from memory while borrowing the VLM for planning and live questions. Only text answers leave the device, and the cloud call is optional and text-only.

The camera loop, the VLM worker and the agent run as separate processes under systemd, so a VLM crash never stops perception, and each can be restarted, profiled and tested on its own.

Five ideas that make it non-trivial

These five decisions are what an interviewer will probe, and each one comes with a number you can measure.

1. **Two-speed perception.** A cheap C++ loop (detector, tracker, depth) runs on every frame, and the VLM runs only on salient events, user questions and alert checks. Proof: VLM calls per hour with and without gating, at the same event recall.
2. **The LLM is a compiler, not an interpreter.** "Tell me if the garage door stays open after 10 pm" is compiled once into a small JSON rule (subject, zone in meters, condition, pickup delay, dropout delay) that a C++ engine checks every frame. No language model sits in the hot loop, rules are unit-testable, and you confirm each compiled rule before it goes live. Proof: rule-check time per frame, and alert precision with and without VLM confirmation.
3. **One resident model, budgeted to the megabyte.** A single 2B VLM is both the eyes (captions, alert checks) and the brain (agent planning through schema-checked JSON tool calls). A priority scheduler serves user questions first, then alert checks, then background captions, and merges

stale captions under load. Proof: per-process memory high-water marks and question latency under load.

4. **Memories anchored in meters.** Depth from the D415 turns each detection into a 3D point, so zones are defined in meters and answers can say "1.4 m left of the door". Two custom CUDA kernels carry it: fused preprocessing straight into the TensorRT input buffer, and a per-box robust depth (histogram median) plus deprojection. Proof: position error in centimeters against tape-measured ground truth, and kernel time against a CPU baseline.
5. **MCP-native, so any agent can use it.** The tools (find object, query events, describe now, measure, create watch rule) are served over MCP's Streamable HTTP transport with token auth. NeMo Agent Toolkit works as an MCP client and Agentforce ships a native one, so the same board plugs into NVIDIA and Salesforce agent stacks. It also mirrors NVIDIA's own JetPack 7.2 pitch: agentic AI on Jetson for robotics, inspection and industrial automation.

Tech stack, with a fallback for every layer

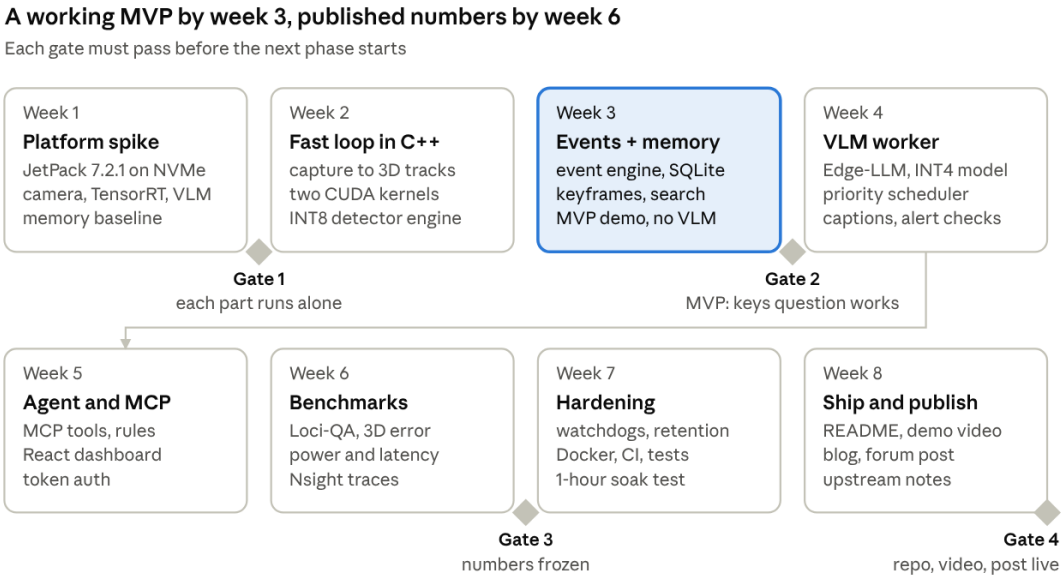
Every layer has a fallback, so no single tool can stall the project. The one rule on JetPack 7.2 is to build from source for the Orin's CUDA architecture (sm_87): a June 2026 setup guide found that containers and wheels built for JetPack 6 crashed, fell back to CPU or produced bad output, while source builds worked.

Layer	Primary choice	Why	Fallback
OS	JetPack 7.2.1 on the NVMe, installed from the ISO on a USB stick	Newest NVIDIA stack and the official Edge-LLM path for Orin	JetPack 6.2.2 with llama.cpp for the VLM
Camera	librealsense, RSUSB build with CUDA on	No kernel patching	Native backend with the kernel patch script
Detector	YOLOE with your own object vocabulary (keys are not a COCO class); ONNX exported on your Mac, INT8 engine built on the board	Open vocabulary, and DeepStream 9.1 documents YOLOE	YOLO11n fine-tuned on your own objects
Tracker	ByteTrack-style tracker in C++	Stable IDs for dwell, move and vanish events	DeepStream NvDCF tracker
Kernels	CUDA C++, profiled in Nsight Systems	Removes the CPU pre/post-processing cost	OpenCV CUDA or VPI 4.1
VLM	Cosmos-Reason2-2B INT4 AWQ on TensorRT	4.3 GB peak in community tests	The same model as GGUF on llama.cpp

Layer	Primary choice	Why	Fallback
	Edge-LLM's C++ runtime		
Rules	C++20 engine over a JSON rule schema, GoogleTest	Deterministic and testable	Python engine at a lower check rate
Memory	SQLite for events, 3D positions and rules; a vector index over captions; keyframes on NVMe	Embedded, zero-ops, easy to inspect	SQLite FTS5 keyword search
Agent and API	Python, FastAPI, MCP Python SDK over Streamable HTTP	Builds on your production RAG and MCP work	Plain REST tools
UI	Small React dashboard: live view, timeline, chat, rules	You already ship React	Server-rendered HTML
Ops	Docker Compose, systemd watchdogs, Prometheus metrics, GitHub Actions CI	What SDE1 interviews probe	Shell scripts and log files

8-week build plan

A demo-able MVP lands at the end of week 3 and frozen benchmark numbers by week 6, so applications can start before the polish is done.



8-week build plan · 8 phases, 4 gates

Read it row by row: weeks 1-4 build the system, weeks 5-8 turn it into proof. What each gate means:

- **Gate 1:** JetPack 7.2.1 boots from the NVMe, realsense-viewer streams depth, and the detector engine and the VLM each run on their own, with a headless memory baseline recorded.
- **Gate 2:** "Where did I leave my keys?" is answered from memory with a 3D position and a keyframe, no VLM involved, and the event engine's unit tests pass in CI. Start applying here with the MVP demo.
- **Gate 3:** every row of the measurement table is filled by a script, and those numbers become the README and resume figures.
- **Gate 4:** the repo, demo video and write-up are public, and the Edge-LLM notes are posted.

The plan assumes roughly 15 hours a week next to applications and LeetCode. With less time, stretch the weeks but keep the gates in order.

What to measure and publish

The numbers are the product: every resume claim will come from this table, measured on your board and published with scripts that reproduce it.

Question	Metric	How
Is perception real-time?	End-to-end FPS; p50 and p99 latency per stage	CUDA events plus an Nsight Systems trace, 10-minute runs
Did INT8 cost accuracy?	mAP on about 300 labeled frames of your room, FP16 vs INT8	Your own labeled set

Question	Metric	How
Is 3D grounding accurate?	Position error (cm) at 0.5-3 m; box-dimension error against a ruler	Tape-measured ground truth
Does the memory answer correctly?	Accuracy on a 100-question "Loci-QA" set over an 8-hour recording	Questions written before seeing answers, graded by script
Does gating pay off?	VLM calls per hour and event recall, gated vs ungated	The same recording replayed both ways
Does VLM confirmation help alerts?	Alert precision and recall, with vs without the VLM check	50 staged events with known labels
Does it fit?	Peak and steady memory per process; swap use	tegrastats during a 1-hour soak test
Does the detector survive VLM load?	Detector p99 latency while the VLM decodes, with and without CUDA stream priorities	Two runs, same workload
What does it cost to run?	Watts, and joules per answered question, at 15 W, 25 W and MAXN SUPER	jtop and tegrastats power readings
Which runtime wins on JetPack 7.2?	Time to first token, tokens/s, peak memory: Edge-LLM vs llama.cpp	Same prompts and image, 5 runs each

The last row doubles as a community contribution: the published Orin Nano VLM numbers above are from JetPack 6.2.2, so JetPack 7.2 results are new data worth posting.

How to pitch it to each company

The project stays the same everywhere; what changes is the layer you lead with and one optional stretch per company.

Company	What entry-level postings ask for	Lead with	Optional stretch
NVIDIA	C++/CUDA inference, TensorRT and Edge-LLM benchmarking, Jetson and robotics	Edge-LLM on the Orin Nano, the two kernels, Nsight traces, the memory budget	Post your JetPack 7.2 build notes and numbers to the TensorRT Edge-LLM repo

Company	What entry-level postings ask for	Lead with	Optional stretch
AMD	Optimizing ML kernels; GPU programming in CUDA or ROCm (ROCm Core NCG)	The kernels and your profiling method	Port both kernels to HIP with hipify and compare
Intel	OpenVINO: accelerating AI frameworks on Intel CPUs, GPUs and NPUs (OpenVINO team)	The ONNX-first model pipeline behind an inference interface	Add an OpenVINO backend and run the detector on an Intel laptop
SEL	C++, real-time firmware, Linux toolchains, CMake/Docker/Jenkins, industrial controls and protocols (Software Engineer)	The deterministic C++ rule engine: pickup and dropout timers, watchdogs, tests	Publish alerts over Modbus TCP or MQTT, tied to your PLC/SCADA work
Cisco	Building AI agents, inter-agent communication protocols, security (early-career AI role)	MCP over Streamable HTTP with auth, the no-video-egress design, telemetry	Store-and-forward alerts that survive a network outage
Salesforce	Architecture, design, implementation and testing in an Agentforce-centered org (AMTS new grad)	Agent tool design, guardrails, human confirmation of compiled rules	Register Loci as an MCP server in Agentforce
IBM	AI engineering on watsonx with Granite models (Entry Level AI Software Engineer)	The model-agnostic agent layer and the evaluation harness	Use a Granite model for cloud escalation
Expedia	End-to-end systems, deployment and monitoring, LLM apps and agentic workflows (SDE I)	API design, the React UI, CI/CD, observability, a runbook	A cloud relay on AWS for remote access

Other Bay Area SDE1 loops score the same fundamentals (system design, clean APIs, tests, observability), and all four are visible in the repo.

Risks and how to de-risk them early

Every risk below has a week-1 test, so a blocker shows up in days rather than weeks.

Risk	Early test	Fallback
Edge-LLM on the Nano still needs the JetPack 6.2.2 workarounds, or the engine build runs out of memory	Run the official Qwen3-4B INT4 tutorial first, then the VLM, with the low-memory export flag	Cosmos-Reason2-2B as GGUF on llama.cpp, Jetson AI Lab's pick for the Nano
No x86 NVIDIA GPU for the Edge-LLM export step (your Mac cannot run it)	Rent an Ampere-or-newer cloud GPU with 24 GB for an afternoon	The llama.cpp path, which needs no export
Prebuilt JetPack 6 containers and wheels misbehave on 7.2	Build everything from source for sm_87	Stay on JetPack 6.2.2
RealSense not detected on JetPack 7.2	RSUSB build and realsense-viewer on day 1	Native backend kernel patch, or JetPack 6.2.2
ISO install fails from your current firmware	Try the ISO first	SDK Manager, which needs an Ubuntu x86 machine
Memory pressure kills the VLM	Headless boot plus a 1-hour tegrastats soak	Shorter context, text-only embeddings, NVMe swap
Boot failures after power cycles on some Orin Nano modules	Install the PCIe overlay fix listed on the JetPack 7.2 page	Shut down cleanly before unplugging
Scope creep	MVP gate at week 3: the keys question works without the VLM	Cut stretch goals, never the benchmarks

Alternatives considered

Loci wins because it covers inference, systems, agents and vision in one demo; each runner-up covers only part of your target list.

Option	Action	Pros	Cons	Best used when
Loci (recommended)	Build the spatial memory agent described here	Hits C++/CUDA, TensorRT, VLMs, agents, MCP and system design at once	Largest scope; memory is tight	You want one project that works for NVIDIA and generalist SDE1 roles

Option	Action	Pros	Cons	Best used when
VLA rover	Mount the Orin on a small rover and let Cosmos Reason 2 drive missions	Strongest physical-AI story	Hardware cost and debugging time; Isaac ROS for JetPack 7.2 is listed as coming soon; similar contest builds exist	You target Isaac and robotics teams only
Inference benchmark study	Compare Edge-LLM, llama.cpp and vLLM across models and power modes	Maps exactly to TensorRT performance roles; cheap	Not a product; a public Orin Nano study already exists	You want a 2-week side project (Loci already includes it)
Multi-camera DeepStream analytics	Four RTSP streams with detection, tracking and zone events	Direct fit for Metropolis and DeepStream	Crowded space; needs several cameras; little agentic depth	You target DeepStream or Metropolis roles specifically
Industrial inspection station	RealSense dimensioning, defect detection and PLC/SCADA alerts	Best fit for SEL and controls roles; reuses your projects	Narrow appeal for Bay Area SWE and AI roles	SEL or industrial automation becomes your top target

Resume lines and visibility

Until Loci ships with measured numbers and a public repo, it stays as the "NVIDIA Jetson Orin" keyword in your skills row, per your own resume rules. Once it ships, it leads your Projects section; pick the two bullets that match each posting, with brackets filled only from your benchmark table:

- Built Loci, an on-device spatial memory agent on Jetson Orin Nano that answers natural-language questions in [X] s at [Y] W with no video leaving the device, using a C++/CUDA pipeline and TensorRT Edge-LLM.
- Cut VLM calls by [A]% at [B]% event recall by gating a 2B VLM behind a TensorRT INT8 detector and a C++ rule engine with pickup and dropout timers.
- Wrote fused CUDA preprocessing and depth-deprojection kernels, cutting per-frame latency from [C] ms to [D] ms (verified in Nsight Systems) and locating objects within [E] cm.
- Exposed the device as an MCP server (Streamable HTTP, token auth) used by NeMo Agent Toolkit and Claude, scoring [F]% on a 100-question memory benchmark.

Visibility, in this order:

1. A public GitHub repo: README with demo GIF, the diagram, benchmark tables, reproduce scripts and a design doc of tradeoffs.
2. A 90-second demo video for LinkedIn and the README.
3. A technical write-up, for example "Fitting eyes, memory and a brain into 8 GB".
4. A post in the Jetson forum's Jetson Projects category, and a submission to NVIDIA's community projects.
5. Your JetPack 7.2 build notes and numbers as a discussion or pull request on TensorRT Edge-LLM.
6. Outreach to Jetson and Edge-LLM engineers with the write-up rather than the resume, then apply with the link in hand.

Sources

Platform and models:

- Jetson Orin Nano Super Developer Kit specifications (NVIDIA)
- JetPack 7.2 downloads and notes (NVIDIA)
- JetPack 7.2.1 released (JetsonHacks)
- NVIDIA Jetson brings agentic AI to the physical world (NVIDIA blog)
- TensorRT Edge-LLM on Jetson (Jetson AI Lab)
- Cosmos Reason 2 2B (Jetson AI Lab)
- DeepStream on JetPack 7.2 (Seeed Studio)
- librealsense on NVIDIA Jetson (RealSense)
- Jetson Orin Nano benchmarks: YOLOv8 and Cosmos-Reason2-2B (community)
- Setting up the Orin Nano Super on JetPack 7.2 (NVIDIA forum, June 2026)
- Cosmos-Reason2-2B running on Jetson Orin Nano (NVIDIA forum)
- Low performance on Jetson Orin Nano Super (NVIDIA forum)
- NeMo Agent Toolkit documentation (NVIDIA)
- Discover MCP for Agentforce (Salesforce Trailhead)
- ERIC, a Cosmos Reason 2 rover on Orin Nano (community)

Job postings (requirements as listed in October 2026 search results; check each posting before applying):

- NVIDIA: ML Inference NCG, TensorRT Performance NCG, Deep Learning Libraries NCG, DevTech AI NCG, Embedded and Robotics FAE NCG, Isaac robotics, 2027 NCG example
- AMD ROCm Core NCG, Intel OpenVINO, SEL Software Engineer, Cisco early-career AI, Salesforce AMTS, IBM Entry Level AI Software Engineer, Expedia SDE I